

Fundamentos de Interpretación y Compilación de Lenguajes de Programación

INFORMACIÓN BÁSICA			
Código y Nombre		750017C - Fundamentos de Interpretación y Compilación de Lenguajes de Programación	
Créditos		3	
Horas de trabajo		Presenciales: 4 horas Trabajo independiente: 5 horas	
Unidad(es) Académica(s)		Escuela de Ingeniería de Sistemas y Computación Facultad de Ingeniería	
Programas Académicos		Ingeniería de Sistemas	
Prerrequisitos		750013C - Fundamentos de Programación Funcional y Concurrente	
Validable		Si	
Habilitable		No	
Tipo de Asignatura		Asignatura Profesional (AP)	
La asignatura favorece la Formación General			
Si			

DESCRIPCIÓN GENERAL DEL CURSO			
En la literatura existen diversos lenguajes de programación compilados e interpretados que se utilizan en la industria y en la educación. En ese sentido, la elección de un lenguaje particular para resolver un problema específico no es una tarea que debe tomarse a la ligera sin comprender las implicaciones que una elección adecuada o inadecuada pueda traer. Por otro lado, el listado de lenguajes de programación es abrumador y tratar de instruirse en cada uno de ellos no es una estrategia aconsejable. Este curso está orientado a comprender y utilizar conceptos fundamentales para la construcción de lenguajes de programación enfocados en tres paradigmas: funcional, imperativo y orientado a objetos. Durante el curso, el estudiante desarrollará habilidades que le permitirán abstraer conceptos fundamentales para la construcción o selección de lenguajes de programación utilizados para resolver problemas específicos. Adicionalmente, se aplicarán estos conceptos en el desarrollo de un lenguaje de programación que el estudiante deberá utilizar para implementar soluciones a problemas planteados en el curso. El curso se fundamenta en tres paradigmas de programación y conceptos asociados, entre ellos: la abstracción de datos como técnica fundamental de programación; formas de ligamiento, visibilidad y alcance de variables; la implementación de interpretadores dirigidos por la sintaxis; ventajas y desventajas de lenguajes interpretados y compilados; el concepto de cerradura (o clausura) para lenguajes funcionales; el concepto de estado y secuenciación para lenguajes imperativos; características			

No de Versión:	**	No. y fecha acta unidad académica donde se aprobó:	(En caso de modificación en la sección "desarrollo del curso" debe actualizarse de lo contrario se mantiene)
Fecha actualización:	**		

de los lenguajes tipados y estrategias utilizadas; el concepto de clase, objeto, herencia y polimorfismo para lenguajes de programación orientados a objetos.

DESARROLLO DEL CURSO		
COMPETENCIA	RESULTADO DE APRENDIZAJE	CONTENIDO
C.E.1: Utilizar los conocimientos fundamentales en teoría de la computación (v.gr. estructuras discretas, complejidad de algoritmos, máquinas abstractas, lógica) en la construcción de sistemas basados en TIC	R.A.1. Utiliza gramáticas independientes del contexto, Analizadores léxicos y sintácticos, en la construcción de interpretadores de lenguajes de programación, para especificar la sintaxis de programas válidos dentro del lenguaje.	Construcción recursiva de datos con métodos de inducción y gramáticas BNF; Gramáticas BNF para la construcción de programas recursivos; nociones de ligadura y alcance de variables; tipos Abstractos de Datos y sus componentes: interfaz e implementación; árbol de sintaxis abstracta; Funciones Parse y Unparse para derivar entre representación concreta y abstracta de una gramática BNF
	R.A.2. Aplica técnicas de Representación de programas y análisis semántico, en la construcción de interpretadores, para traducir y ejecutar programas.	Ambientes para el uso de ligadura de variables; árboles de sintaxis abstracta; comportamiento de los condicionales en un lenguaje de programación interpretado; concepto de clausura para el manejo y comportamiento de procedimientos y recursión en un lenguaje de programación funcional; concepto de estado asociado al manejo de ambientes para el uso de variables en lenguajes de programación imperativos; ligadura y asignación de variables; paso de parámetros por valor y por referencia; lenguajes de programación tipados - chequeo e inferencia de tipos; lenguajes de programación orientados a objetos a través de estrategias para la representación de clases y objetos.
C.E.3: Seleccionar y utilizar diferentes paradigmas y lenguajes de programación al construir sistemas basados en TIC.	R.A.3. Comprende las diferencias entre compilación e interpretación, identificando los beneficios y limitaciones de rendimiento y manejo de memoria, para seleccionar lenguajes de programación adecuados a las características del problema a resolver.	Componentes y características de un interpretador y un compilador (lenguajes compilados vs interpretados).
C.G.2: Utilizar un pensamiento crítico y creativo que le permite aprender de forma autónoma y permanente.	R.A.4. Construye un interpretador, a partir de una gramática básica, que implementa diversos conceptos aprendidos sobre lenguajes de programación como ligadura de variables, manejo de procedimientos, recursión, tipos de datos, representación de objetos, entre otros.	Programas que recorren e interpretan árboles de sintaxis abstracta.

METODOLOGÍA

El curso se desarrolla en clases semanales de 4 horas. En las clases se aborda al menos un caso de estudio orientado a introducir nuevos conceptos y construir un lenguaje de programación enfocado a los resultados de aprendizaje que se quieren alcanzar. Dentro del proceso formativo se considera como estrategia de aprendizaje la resolución de problemas a través del uso de talleres y el proyecto del curso donde grupos de estudiantes deberán construir un lenguaje de programación. Esto les permitirá poner en práctica los conceptos vistos en clase y también explorar diferentes estrategias de diseño de lenguajes de programación. Durante las diferentes actividades del proyecto y en fechas específicas, cada equipo debe hacer entregas con un alcance definido.

RECURSOS DE APOYO

El principal recurso de apoyo es el campus virtual donde se encuentra desplegado el curso con todos sus medios y materiales. Dentro del proceso formativo se considera el uso de plataformas interactivas en el aula (tipo encuestas online como Socrative) que permitan revisar conceptos aprendidos, la revisión de videos de las unidades que componen el curso y las clases magistrales. Para el desarrollo efectivo del curso se requiere de un salón de clase con acceso a internet, con puestos individuales de trabajo y que esté provista de video beam.

EVALUACIÓN DEL CURSO

La evaluación del curso incluye tres talleres y un proyecto que se realizarán en grupo, dos parciales que se realizarán de forma individual. Los porcentajes asignados para cada resultado de aprendizaje son los siguientes:

Resultado de aprendizaje	%	Posibles actividades evaluativas
R.A.1	40	Parcial 1, Talleres
R.A.2	50	Parcial 1, Parcial 2, Talleres, Proyecto
R.A.3	5	Talleres
R.A.4	5	Proyecto
TOTAL	100	

BIBLIOGRAFÍA

Friedman D.P., Wand M. and Haynes C. Essentials of Programming Languages, The MIT Press, Third Edition, 2009.

Stansifer, R. The Study of Programming Languages, Prentice Hall, 1995, Paradigma Imperativo y otros.

Van Roy, P. and Haridi, S. Concepts, Techniques and Models of Computer Programming, MITPress, 2003.

Appel, Andrew W. Modern compiler implementation in C. Cambridge university press, 2004.

Clinton L. Jeffery. Build Your Own Programming Language: A programmer's guide to designing compilers, interpreters, and DSLs for solving modern computing problems. Packt Publishing, 2021.

Thorsten. B. Writing An Interpreter In Go, Thorsten. B, 2018

Thorsten. B. Writing a Compiler In Go, Thorsten. B, 2018